



**Universiti
Putra
Malaysia**

Database Principles CCS3400

Non-Relational Databases (NoSQL)

Learning Objectives



Understand Different NoSQL Data Models



Compare SQL vs NoSQL



Real World application of NoSQL

Introduction

NoSQL (Not Only SQL) databases are a category of database management systems **designed to handle large volumes of unstructured, semi-structured**, or rapidly changing data



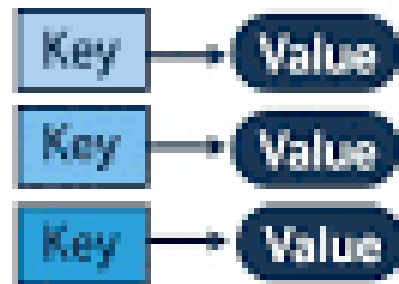
Unlike traditional relational databases that use fixed table schemas and SQL for queries, NoSQL databases offer flexible data models, horizontal scalability, and high performance for modern web-scale applications.



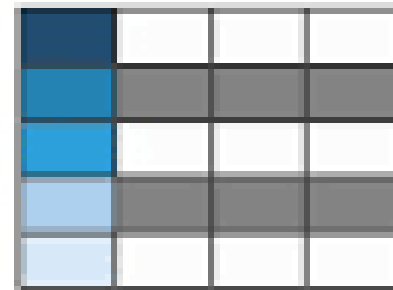
These databases are particularly useful in distributed data environments and are commonly used in big data, real-time analytics, mobile apps, and content management systems.

NoSQL

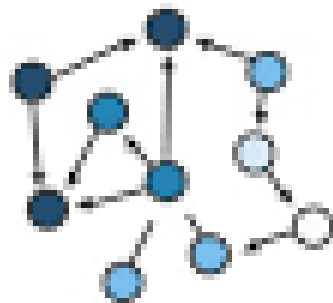
Key-Value



Column-Family



Graph



Document



Types & Categories of NoSQL Databases

NoSQL databases are categorized based on how they store and manage data. Common types include:

Type	Description	Example Database	Use Case Example
Document-based	Stores data in JSON, BSON, or XML documents. Flexible schema.	MongoDB, CouchDB	E-commerce product catalogs where each product may have different attributes.
Key-Value Store	Data is stored as key-value pairs. Simple and highly scalable.	Redis, DynamoDB	Session storage in web applications.
Column-Family Store	Stores data in rows and columns but groups columns into families.	Apache Cassandra, HBase	Time-series data like sensor readings or logs.
Graph-based	Uses graph structures with nodes, edges, and properties.	Neo4j, ArangoDB	Social network data, recommendation systems.

Applications of NoSQL Databases

Application Area	Explanation	Example
Real-time Big Data Analytics	NoSQL systems can process massive data sets in real time across clusters.	Cassandra for log analytics or clickstream data
Content Management Systems (CMS)	Content with variable structure (articles, media) suits document-based NoSQL.	MongoDB in platforms like WordPress alternatives
Mobile & IoT Data Management	NoSQL handles massive writes and semi-structured sensor data well.	Firebase for mobile app backend; Couchbase for IoT sensors
Personalization & Recommendation Engines	Graph databases model relationships and patterns.	Neo4j for personalized content recommendations
High-Speed Caching	Key-value stores offer ultra-fast retrieval.	Redis for storing frequently accessed data like API tokens

Document- based database

Beers Table

1167	Ale C	Miller	578
3424	Beerio	Ians	348
5612	Amstel	Antel	121
2489	Colt's	BeerCo	98

Beers Documents



Document-based

- Document-based databases store data in the form of **documents**, typically using formats like **JSON**, **BSON**, or **XML**. Each document is a self-contained data unit that can contain nested structures such as arrays or sub-documents.
- Unlike relational databases, document stores do not require a predefined schema. This flexibility allows documents in the same collection (similar to a table) to have different fields

◆ Structure Example (JSON Format)

json

```
{  
  "product_id": "P1023",  
  "name": "Wireless Mouse",  
  "price": 79.99,  
  "in_stock": true,  
  "specs": {  
    "brand": "LogiTech",  
    "color": "Black",  
    "connectivity": "Bluetooth"  
  }  
}
```

Document-based: Key Features

- **Flexible Schema:** No need to define table structure in advance.
- **Nested Documents:** Allows storing complex hierarchical data easily.
- **High Performance:** Optimized for read-heavy and document-centric queries.
- **Indexing Support:** Indexes can be created on fields to enhance performance.

Popular Document Databases

Database	Description
MongoDB	Most popular document store. Uses BSON format internally.
CouchDB	Uses JSON and supports RESTful HTTP API for access.
RethinkDB	Real-time updates to query results. Good for reactive applications.
Amazon DocumentDB	Managed document database service compatible with MongoDB.

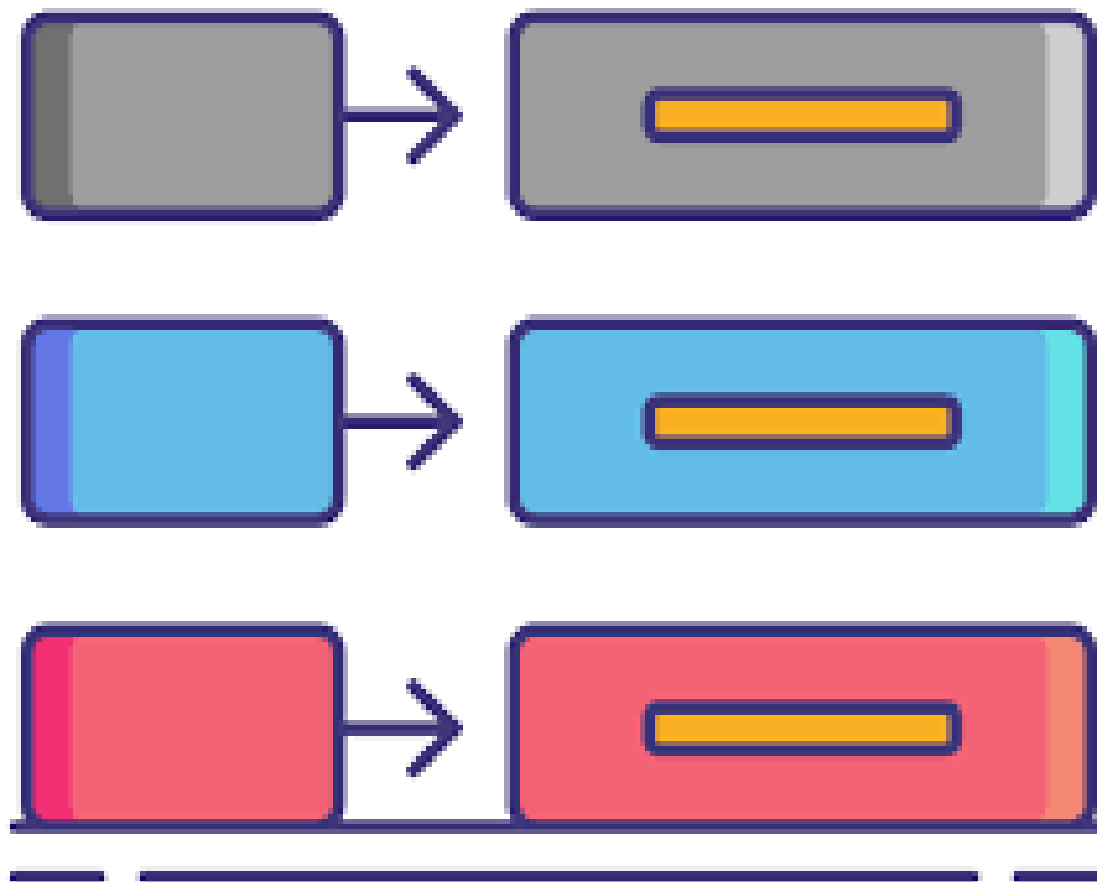
Document-based Use Case

Scenario	Why Document DB is Suitable
E-commerce Product Catalog	Products have varied attributes; schema flexibility helps.
Content Management Systems (CMS)	Articles, images, and metadata differ in structure.
Mobile Applications	JSON aligns with data formats used in APIs and mobile apps.
User Profiles	Each user may have different preferences, settings, and history.

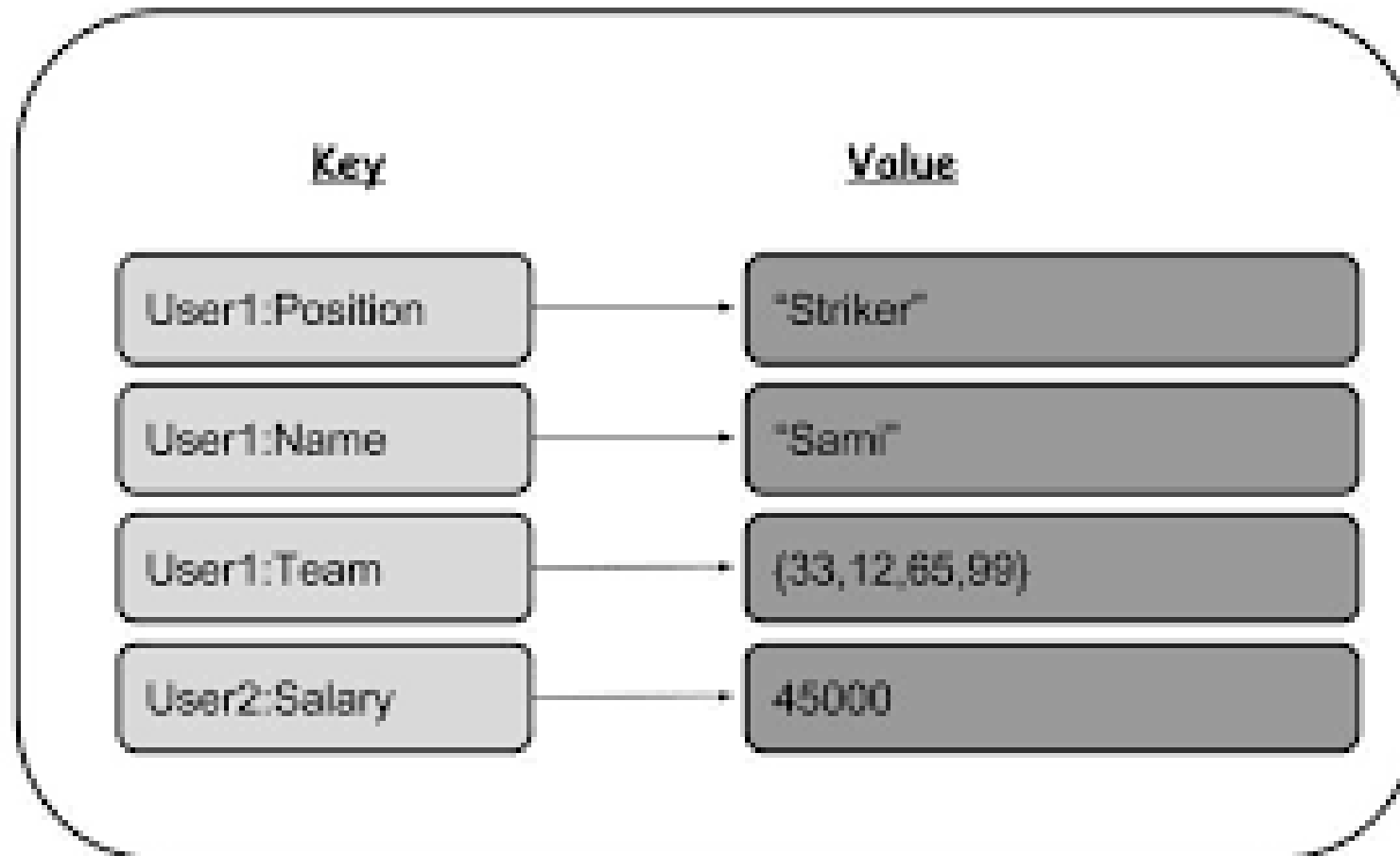
Advantages and Disadvantage of Document Database

Advantages	Disadvantages
Flexible schema – no need to define strict tables/columns in advance	No strong schema enforcement – risk of inconsistent data structure
Natural JSON format – easy integration with modern apps (web/mobile)	Weaker support for complex relationships (no advanced joins)
Fast read/write performance for large volumes of data	Data duplication due to denormalization → storage overhead
Horizontal scalability (sharding across servers)	Data consistency challenges – no built-in foreign key constraints
Handles unstructured/semi-structured data well (logs, user profiles, catalogs)	Limited transaction support compared to SQL (though improving)
Developer-friendly – easier to evolve data models over time	Vendor lock-in / lack of standardization (query languages differ)
Good for hierarchical/nested data (arrays, objects inside documents)	Complex queries/aggregations can be less efficient than SQL

Key-Value
Store
(NoSQL)



Key-Value Store (NoSQL)





What is a Key-Value Store (NoSQL)

- **Definition**
- A **Key-Value Store** is the simplest type of NoSQL database. It stores data as a collection of **key–value pairs**, where:
- The **key** is a unique identifier (like a variable name).
- The **value** is the data associated with that key (can be a string, number, JSON, etc.).
- There is **no fixed schema**, and the database doesn't need to know the structure of the value—it just retrieves or stores it based on the key.

◆ Structure Example

json

 Copy  Edit

```
{  
  "user_001": "John Doe",  
  "user_002": "Jane Smith",  
  "session_123": {  
    "user": "user_001",  
    "login_time": "2025-08-04T09:00:00Z",  
    "ip": "192.168.1.1"  
  }  
}
```

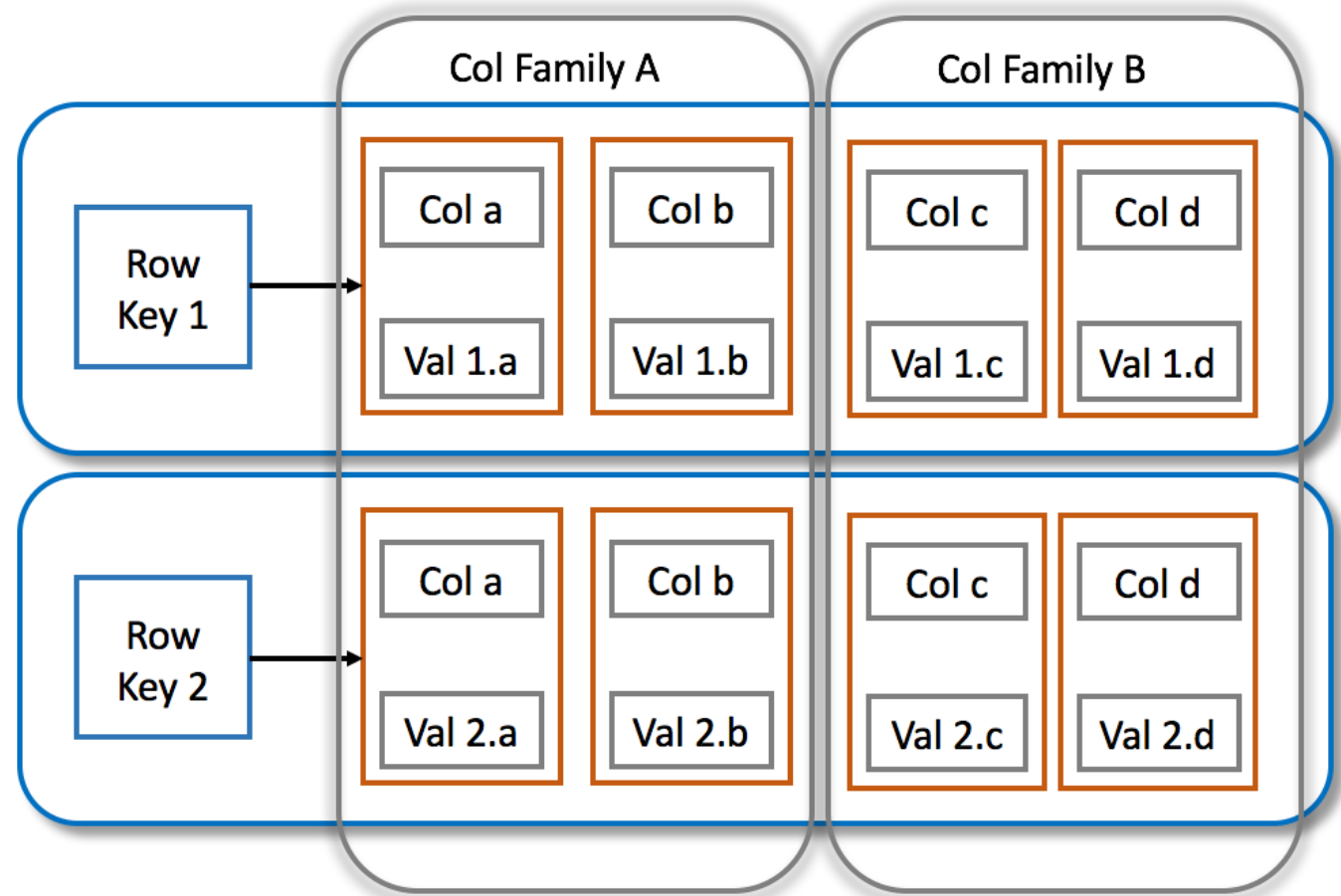


Each key directly maps to a value. The value can be a simple string or a complex object.

Popular Key-Value Databases

Database	Description
Redis	In-memory, supports TTL, persistence, and advanced data structures like sets, lists
Amazon DynamoDB	Fully managed, scalable key-value and document database
Riak KV	Distributed system designed for fault tolerance
Berkeley DB	Embedded key-value storage engine by Oracle

Column-Family Store (NoSQL)





What is a Column-Family Store

- A **Column-Family Store** is a type of NoSQL database that organizes data into **rows and columns**, similar to relational databases, **but with a crucial difference**: columns are grouped into **families**, and each row doesn't need to have the same columns.
- Each **column family** is stored together, allowing efficient **read and write operations** for large-scale datasets. These databases are highly scalable and are often used in distributed systems.

◆ Structure Example

Row Key	user:name	user:email	order:total
001	Alice	alice@example.com	123.45
002	Bob	bob@example.com	67.89

In this example:

- Columns are grouped into **families**: `user:*` and `order:*`
- Each row can have a **different set of columns** within a family

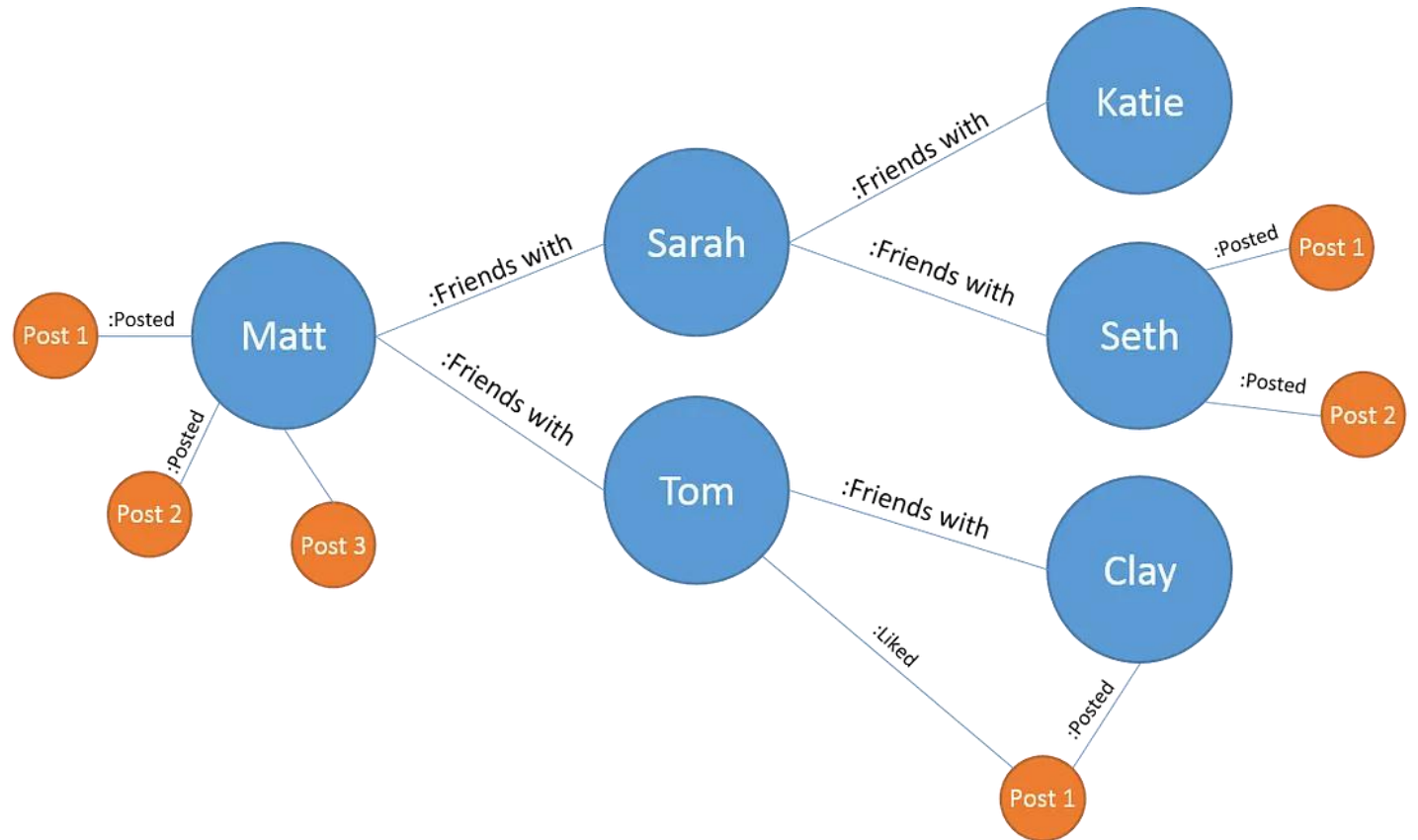
Column families store-Key Features

- **Column families** store related columns together on disk
- Excellent performance for **wide and sparse data**
- Optimized for **write-heavy workloads**
- Data is accessed via **row keys**, often sorted by timestamp or lexicographically
- Supports **denormalized designs** (reduces need for joins)

Popular Column-Family Databases

Database	Description
Apache Cassandra	Highly available, partition-tolerant DB with tunable consistency
Apache HBase	Built on top of Hadoop HDFS, supports real-time read/write
ScyllaDB	Drop-in replacement for Cassandra with better performance
Hypertable	Open-source HBase alternative inspired by Google's Bigtable

Graph Database





What is a Graph Database?

- A **Graph Database** is a type of NoSQL database designed to store and analyze **relationships** between data.
 - **Nodes** represent entities (e.g., Person, Product, Course).
 - **Edges** represent relationships between entities (e.g., FRIEND, LIKES, BUYS).
 - **Properties** are attributes that can be stored on nodes and edges.
- Popular systems: **Neo4j, ArangoDB, Amazon Neptune**



Example Representation (Social Network Scenario)

SQL (Relational Database)

Would require **multiple tables**:

sql

```
CREATE TABLE person (id INT, name VARCHAR(50), age INT);  
CREATE TABLE friendship (person1 INT, person2 INT);
```

Graph Database (Neo4j with Cypher)

c

```
CREATE (a:Person {name:'Ali', age:22})  
CREATE (s:Person {name:'Siti', age:20})  
CREATE (j:Person {name:'John', age:23})  
CREATE (a)-[:FRIEND]->(s)  
CREATE (s)-[:FRIEND]->(j)  
CREATE (a)-[:LIKES]->(:Course {name:'Database'})
```



Advantages of Graph Databases

- Queries about **relationships** are much simpler.
(Example: “Who are Ali’s friends of friends?”)
- No need for complex joins across multiple tables.
- Very useful for **social networks, recommendation systems, fraud detection, knowledge graphs**.



Example Query (Neo4j – Cypher)

Find Ali's friends of friends:

cypher

```
MATCH (a:Person {name:'Ali'})-[:FRIEND]->(friend)-[:FRIEND]->(fof)  
RETURN fof.name;
```

Graph Database (Neo4j with Cypher)

C

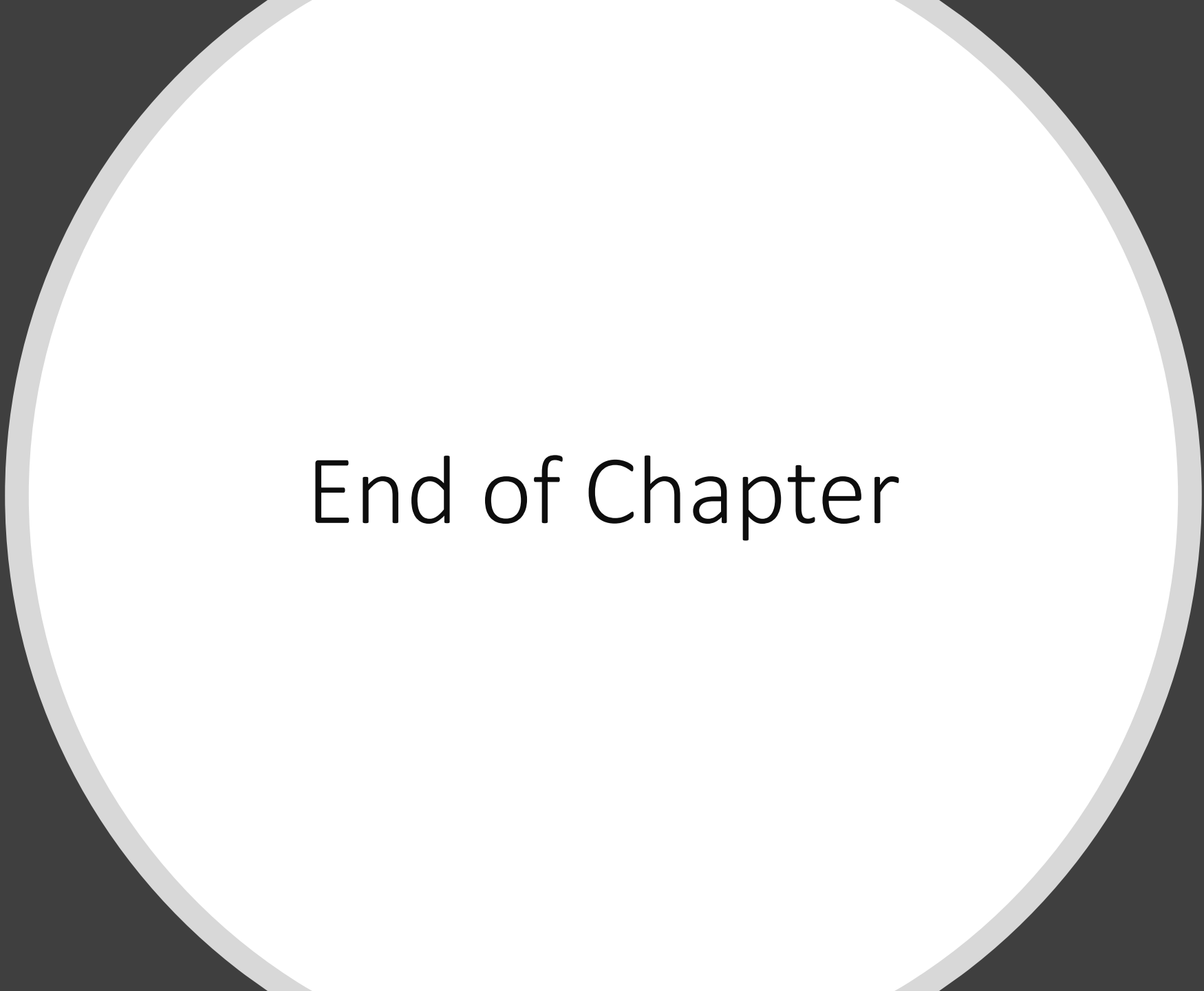
```
CREATE (a:Person {name:'Ali', age:22})
CREATE (s:Person {name:'Siti', age:20})
CREATE (j:Person {name:'John', age:23})
CREATE (a)-[:FRIEND]->(s)
CREATE (s)-[:FRIEND]->(j)
CREATE (a)-[:LIKES]->(:Course {name:'Database'})
```

SQL Constraints vs. NoSQL Advantages

SQL Constraints	NoSQL Advantages
Fixed schema – altering structure requires table changes	Flexible schema – easily adapts to changing data models
Vertical scaling (adding more power to one server)	Horizontal scaling (distributed architecture with multiple servers)
Complex JOINS may reduce performance at scale	Optimized for specific data access patterns (e.g., documents or graphs)
ACID transactions focus	Eventual consistency, BASE model is more scalable and performant in distributed systems

Key Takeaways – NoSQL Databases

- NoSQL provides **flexible, scalable, and high-performance** alternatives to relational databases.
- Four main types: **Document, Key-Value, Column-Family, Graph**, each suited to different use cases.
- Best for **big data, real-time analytics, social networks, IoT, and e-commerce**.
- Trade-off: **Flexibility & scalability** vs. **strict schema & strong relationships** of SQL.



End of Chapter